
Objetivo do Material

Sumário

Objetivo do Material	3
Como usar este playbook	4
Mapa dos Estágios	5
Passos Práticos	6
Passo 1 — O Arsenal Gratuito do Analista	6
Passo 2 — O Poder dos Webhooks	9
Passo 3 — Automação Pós-Venda na Prática	14
Passo 4 — Alertas Executivos	17
Checklist Mestre	23

Objetivo do Material

Apresentar a promessa de construir um motor de inteligência financeira com custo zero — sem depender de TI, sem orçamento, com ferramentas que já existem.

Como usar este playbook

Você é o **Engenheiro de Automação Financeira**. Cada passo é um card independente com sete partes: objetivo, pré-requisito, entregas, execução, gate de verificação, critério de conclusão e armadilhas.

Este documento **não repete a teoria** do livro. Quando precisar do porquê, siga a referência cruzada do card para o capítulo correspondente.

Mapa dos Estágios

#	Estágio	Passos
1	Bancada	1, 2
2	Ferramenta	3, 4

Passos Práticos

Passo 1 — O Arsenal Gratuito do Analista

Estágio: Bancada · **Origem:** Cap. 1 — O Arsenal Gratuito do Analista

① Objetivo do passo

Mapear e configurar as ferramentas de IA gratuitas disponíveis (Google AI Studio, Hugging Face) para análise financeira.

② Pré-requisito

Nenhum — este é o ponto de partida

③ Entregas

- `nlptown/bert-base-multilingual-uncased-sentiment`

④ Execução

Configurando o Google AI Studio

```
# configuração_inicial.py
# Script para testar a conexão com o Google AI Studio

import requests
import json

# Sua chave de API (substitua pela sua)
API_KEY = "<sua-chave-api-gemini>"
BASE_URL = "https://generativelanguage.googleapis.com/v1beta/models"

def listar_modelos_disponiveis():
    """Lista os modelos Gemini disponíveis no tier gratuito"""
    url = f"{BASE_URL}?key={API_KEY}"
    resposta = requests.get(url)

    if resposta.status_code == 200:
        modelos = resposta.json().get("models", [])
        print("Modelos disponíveis:")
        for modelo in modelos:
            nome = modelo.get("name", "N/A")
            capacidades = modelo.get("supportedGenerationMethods", [])
            print(f"  - {nome}: {capacidades}")
        return modelos
```

```

else:
    print(f"Erro {resposta.status_code}: {resposta.text}")
    return None

def analisar_faturamento(dados_financeiros):
    """Envia dados de faturamento para o Gemini e recebe análise"""
    modelo = "gemini-1.5-flash" # Tier gratuito: 15 RPM
    url = f"{BASE_URL}/{modelo}:generateContent?key={API_KEY}"

    prompt = f"""
    Analise os seguintes dados de faturamento de clínica odontológica e retorne:
    1. Total faturado no período
    2. Ticket médio por procedimento
    3. Procedimento mais lucrativo
    4. Sugestão de otimização

    Dados: {json.dumps(dados_financeiros, ensure_ascii=False)}
    """

    payload = {
        "contents": [{"parts": [{"text": prompt}]}],
        "generationConfig": {
            "temperature": 0.3,
            "maxOutputTokens": 1024
        }
    }

    headers =

```

Configurando o Hugging Face para classificação de feedbacks

```

# classificador_feedbacks.py
# Script para classificar feedbacks de clientes usando Hugging Face

import requests
import json
import csv
from datetime import datetime

# Configuração do Hugging Face
HF_API_URL = "https://api-inference.huggingface.co/models/nlptown/bert-base-multilingual-uncased-sentiment"
HF_TOKEN = "<seu-huggingface-token>" # Opcional para modelos gratuitos

def classificar_feedback(texto_feedback):
    """Classifica um feedback em sentimento (1-5 estrelas)"""
    headers = {"Authorization": f"Bearer {HF_TOKEN}"} if HF_TOKEN else {}

    payload = {"inputs": texto_feedback}
    resposta = requests.post(HF_API_URL, headers=headers, json=payload)

    if resposta.status_code == 200:
        resultado = resposta.json()
        # O modelo retorna array de labels com scores
        if isinstance(resultado, list) and len(resultado) > 0:
            scores = resultado[0]
            # Encontra o label com maior score
            melhor = max(scores, key=lambda x: x["score"])

```

```

label = melhor["label"] # Ex: "5 stars", "1 star"
score = melhor["score"]

# Mapeia para classificação simplificada
estrelas = int(label.split()[0])
if estrelas >= 4:
    classificacao = "promotor"
elif estrelas == 3:
    classificacao = "passivo"
else:
    classificacao = "detrator"

return {
    "texto": texto_feedback,
    "estrelas": estrelas,
    "classificacao": classificacao,
    "confianca": round(score, 3)
}

```

A stack completa em ação: workflow N8N de exemplo

```

# workflow_exemplo.json
# Workflow N8N que conecta Google Sheets ao Google AI Studio
{
  "name": "Análise Financeira Automatizada",
  "nodes": [
    {
      "parameters": {
        "pollTimes": {
          "item": [{"mode": "everyMinute"}]
        },
        "documentId": {"__rl": true, "value": "SUA_PLANILHA_ID"},
        "sheetName": {"__rl": true, "value": "Dados Brutos"}
      },
      "name": "Google Sheets - Ler Dados",
      "type": "n8n-nodes-base.googleSheets",
      "typeVersion": 4,
      "position": [250, 300]
    },
    {
      "parameters": {
        "conditions": {
          "options": {"caseSensitive": true, "leftValue": "", "typeValidation": "strict"},
          "conditions": [
            {
              "id": "condicao-faturamento",
              "leftValue": "={{ \\$json.faturamento }}",
              "rightValue": "10000",
              "operator": {"type": "number", "operation": "gt"}
            }
          ]
        }
      },
      "name": "IF - Faturamento > 10k",
      "type": "n8n-nodes-base.if",
      "typeVersion": 2,
      "position": [480, 300]
    },
  ],
}

```

```
{
  "parameters": {
    "method": "POST",
    "url": "=https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-
flash:generateContent?key={{ \\$env.GEMINI_API_KEY }}",
    "sendBody": true,
    "bodyParameters": {
      "parameters": [
        {
          "name": "contents",
          "value": "=[{\"parts\": [{\"text\": \"Analise este dado financeiro e retorne
um resumo executivo em 3 linhas: {{ JSON.stringify(\\$json) }}\"}]]]"
        }
      ]
    }
  }
}
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Você acabou de montar sua bancada
- ☐ As ferramentas estão afiadas
- ☐ Mas antes de sair ligando tudo
- ☐ Imagine que você é o responsável financeiro de uma rede de 3 clínicas odontológicas
- ☐ O dono pede um relatório consolidado de faturamento do trimestre
- ☐ A armadilha aqui é sutil: a tentação de automatizar tudo de uma vez
- ☐ Você vê o potencial

⑦ Armadilhas

- (a completar)

Passo 2 — O Poder dos Webhooks

Estágio: Bancada · **Origem:** Cap. 2 — O Poder dos Webhooks

① Objetivo do passo

Ensinar o leitor a entender e configurar webhooks para conectar planilhas financeiras a serviços externos sem programar.

② Pré-requisito

Passo 1 concluído

③ Entregas

- (a completar)

④ Execução

Construindo um webhook receiver com Express (Node.js)

```
// webhook_receiver.js
// Mini-servidor que recebe webhooks POST e retorna status 200

const express = require('express');
const app = express();
const PORT = 3000;

// Middleware para parsear JSON
app.use(express.json());

// Banco de dados em memória (para demonstração)
const webhookLog = [];

// Endpoint principal - recebe webhooks
app.post('/webhook/recebido', (req, res) => {
  const timestamp = new Date().toISOString();
  const dados = req.body;
  const headers = req.headers;

  // Registra o webhook recebido
  const registro = {
    id: webhookLog.length + 1,
    timestamp: timestamp,
    origem: headers['x-webhook-source'] || 'desconhecido',
    dados: dados,
    processado: false
  };

  webhookLog.push(registro);

  console.log(`[${timestamp}] Webhook recebido de: ${registro.origem}`);
  console.log(`  Dados: ${JSON.stringify(dados, null, 2)}`);

  // Retorna status 200 para o disparador saber que deu certo
  res.status(200).json({
    status: 'sucesso',
    mensagem: 'Webhook recebido e registrado',
    id: registro.id,
    timestamp: timestamp
  });
});

// Endpoint para consultar webhooks recebidos
app.get('/webhook/log', (req, res) => {
  res.json({
    total: webhookLog.length,
    registros: webhookLog.slice(-10) // Últimos 10
  });
});

// Endpoint de health check
app.get('/health', (req, res) => {
```

```
    res.json({ status: 'ok', uptime: process.uptime() });
  });

// Inicia o servidor
app.listen(PORT, () => {
  console.log(`\n=== WEBHOOK RECEIVER RODANDO ===`);
  console.log(`Porta: \`${PORT}\``);
  console.log(`Endpoint: http://localhost:\`${PORT}\`/webhook/recebido`);
});
```

Testando com curl

```
# Teste 1: Enviar um webhook simulando dados de faturamento
curl -X POST http://localhost:3000/webhook/recebido \
-H "Content-Type: application/json" \
-H "X-Webhook-Source: google-sheets" \
-d '{
  "evento": "nova_venda",
  "clinica": "Odonto Premium",
  "valor": 7500.00,
  "procedimento": "Clareamento + Facetas",
  "dentista": "Dr. Carlos",
  "data": "2026-08-08"
}'

# Teste 2: Enviar dados de pagamento
curl -X POST http://localhost:3000/webhook/recebido \
-H "Content-Type: application/json" \
-H "X-Webhook-Source: sistema-pagamento" \
-d '{
  "evento": "pagamento_recebido",
  "cliente": "Maria Silva",
  "valor": 2800.00,
  "forma": "PIX",
  "referencia": "FAT-2026-0847"
}'

# Teste 3: Consultar log de webhooks recebidos
curl http://localhost:3000/webhook/log
```

Configurando o N8N para receber e processar webhooks

```
{
  "name": "Webhook Financeiro - Processamento Automatizado",
  "nodes": [
    {
      "parameters": {
        "httpMethod": "POST",
        "path": "financeiro",
        "responseMode": "responseNode",
        "options": {}
      },
      "name": "Webhook - Receber Dados",
      "type": "n8n-nodes-base.webhook",
      "typeVersion": 2,
      "position": [250, 300],
      "webhookId": "webhook-financeiro-001"
    }
  ]
}
```

```

    },
    {
      "parameters": {
        "conditions": {
          "options": {"caseSensitive": true, "leftValue": "", "typeValidation":
"strict"},
          "conditions": [
            {
              "id": "condicao-venda",
              "leftValue": "={{ \\$json.body.evento }}",
              "rightValue": "nova_venda",
              "operator": {"type": "string", "operation": "equals"}
            }
          ]
        }
      },
      "name": "IF - É Venda?",
      "type": "n8n-nodes-base.if",
      "typeVersion": 2,
      "position": [480, 300]
    },
    {
      "parameters": {
        "conditions": {
          "options": {"caseSensitive": true, "leftValue": "", "typeValidation":
"strict"},
          "conditions": [
            {
              "id": "condicao-valor",
              "leftValue": "={{ \\$json.body.valor }}",
              "rightValue": "5000",
              "operator": {"type": "number", "operation": "gt"}
            }
          ]
        }
      },
      "name": "IF - Venda > 5k?",
      "type": "n8n-nodes-base.if",
      "typeVersion": 2,
      "position": [710, 250]
    },
    {
      "parameters": {
        "method": "POST",
        "url": "=ht

```

Estrutura de planilha para automação

```

# setup_planilha.py
# Script para criar a estrutura de abas no Google Sheets via API

from google.oauth2 import service_account
from googleapiclient.discovery import build
import json

# Configuração de autenticação
SCOPES = ['https://www.googleapis.com/auth/spreadsheets']
SERVICE_ACCOUNT_FILE = 'credenciais-gsheets.json' # Baixe do Google Cloud

```

```
def criar_estrutura_planilha(spreadsheet_id):
    """Cria as abas necessárias para automação"""
    creds = service_account.Credentials.from_service_account_file(
        SERVICE_ACCOUNT_FILE, scopes=SCOPES
    )
    service = build('sheets', 'v4', credentials=creds)
    sheets = service.spreadsheets()

    # Estrutura das abas
    abas_config = [
        {
            "titulo": "Dados Brutos",
            "headers": ["Data", "Clinica", "Procedimento", "Dentista", "Valor", "Status"]
        },
        {
            "titulo": "Processados",
            "headers": ["Data", "Tipo", "Descricao", "Resultado", "Confianca"]
        },
        {
            "titulo": "Alertas",
            "headers": ["Data", "Tipo Alerta", "Mensagem", "Destinatario", "Enviado"]
        },
        {
            "titulo": "Config",
            "headers": ["Parametro", "Valor", "Descricao"],
            "dados": [
                ["limiar_venda_grande", "5000", "Valor em R\\$ que dispara alerta"],
                ["limiar_estoque_baixo", "10", "Quantidade mínima em estoque"],
                ["frequencia_alerta", "diario", "Frequência de relatórios"],
                ["whatsapp_destinatario", "+5511999999999", "Número do diretor"]
            ]
        }
    ]
```

⑤ Verificação / Gate

```
curl http://localhost:3000/webhook/log
```

⑥ Feito quando...

- ☐ Você configurou o webhook receiver
- ☐ Agora vamos ver o que acontece quando a teoria encontra a prática —
- ☐ Imagine que você é o analista de uma clínica odontológica que fatura R\$ 180 mil por mês
- ☐ O dono quer ser alertado no WhatsApp sempre que uma venda de R\$ 5 mil ou mais acontecer
- ☐ Você monta o workflow: planilha alimenta webhook
- ☐ O primeiro dia em produção revela a verdade
- ☐ A recepcionista cadastra 3 vendas simultâneas

⑦ Armadilhas

- (a completar)

Passo 3 — Automação Pós-Venda na Prática

Estágio: Ferramenta · **Origem:** Cap. 3 — Automação Pós-Venda na Prática

① Objetivo do passo

Criar um fluxo visual com N8N para automatizar pesquisa de satisfação (NPS) e tabular feedbacks dos dentistas automaticamente.

② Pré-requisito

Passo 2 concluído

③ Entregas

- (a completar)

④ Execução

Instalando o N8N self-hospedado com Docker

```
# docker-compose.yml
# Configuração completa para N8N self-hospedado com persistência

version: '3.8'

services:
  n8n:
    image: n8nio/n8n:latest
    container_name: n8n-oficina
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      # Configurações básicas
      - N8N_BASIC_AUTH_ACTIVE=true
      - N8N_BASIC_AUTH_USER=admin
      - N8N_BASIC_AUTH_PASSWORD=sua_senha_forte_aqui
      - N8N_HOST=localhost
      - N8N_PORT=5678
      - N8N_PROTOCOL=http

      # Webhook URL pública (para receber respostas do WhatsApp)
      - WEBHOOK_URL=https://seu-dominio.com.br/

      # Variáveis de ambiente para integrações
      - GEMINI_API_KEY=sua_chave_gemini
      - GOOGLE_SHEETS_CREDENTIAL_ID=credencial-gsheets

      # Configurações de timezone
      - GENERIC_TIMEZONE=America/Sao_Paulo
      - TZ=America/Sao_Paulo
```

```

# Persistência de dados
- N8N_DEFAULT_BINARY_DATA_MODE=filesystem

volumes:
  # Dados persistidos: workflows, credenciais, histórico
  - n8n_data:/home/node/.n8n
  # Arquivos temporários de binary data
  - n8n_binary:/home/node/.n8n/binaryData

networks:
  - oficina-network

# PostgreSQL para persistência robusta (opcional mas recomendado)
postgres:
  image: postgres:15
  container_name: n8n-postgres
  restart: unless-stopped
  environment:
    - POSTGRES_DB=n8n
    - POSTGRES_USER=n8n
    - POSTGRES_PASSWORD=n8n_senha_forte
  volumes:
    - postgres_data:/var/lib/postgresql/data
  networks:
    - oficina-network

volumes:
  n8n_data:
  n8n_binary:
  postgres_data:

networks:

```

Workflow NPS completo: pipeline do disparo à tabulação

```

{
  "name": "Pipeline NPS - Automação Completa",
  "nodes": [
    {
      "parameters": {
        "rule": {
          "interval": [{"field": "cronExpression", "expression": "0 9 * * 1"}]
        }
      },
      "name": "Schedule - Segunda 9h",
      "type": "n8n-nodes-base.scheduleTrigger",
      "typeVersion": 1.2,
      "position": [200, 300]
    },
    {
      "parameters": {
        "documentId": {"__rl": true, "value": "SUA_PLANILHA_ID"},
        "sheetName": {"__rl": true, "value": "Pacientes NPS"},
        "filtersUI": {
          "filters": [
            {

```

```

        "lookupColumn": "data_procedimento",
        "lookupValue": "={{ {{ $now.minus({days: 7}).toISOString() }}"
    }
  ]
},
{
  "name": "Google Sheets - Pacientes da Semana",
  "type": "n8n-nodes-base.googleSheets",
  "typeVersion": 4,
  "position": [430, 300]
},
{
  "parameters": {
    "method": "POST",
    "url": "={{ {{ $env.EVOLUTION_API_URL }}}/message/sendText/{{ {{ $env.EVOLUTION_INSTANCE }}"
    "authentication": "genericCredentialType",
    "genericAuthType": "httpHeaderAuth",
    "sendBody": true,
    "specifyBody": "json",
    "jsonBody": "={{ {{ $json.whatsapp }}"
    "text": "Olá {{ {{ $json.nome }}"
    "emoji": "😊"
    "avalie de 1 a 10 o atendimento da clínica:
    1 - Péssimo
    5 - Regular
    10 - Excelente
    Responda apenas com o número."
  },
  "name": "Evolution API - Enviar NPS",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 4,
  "position": [660, 300]
},
{

```

Script de relatório NPS consolidado

```

# relatorio_nps.py
# Gera relatório consolidado a partir dos dados do Google Sheets

import json
from datetime import datetime, timedelta
from google.oauth2 import service_account
from googleapiclient.discovery import build

SCOPES = ['https://www.googleapis.com/auth/spreadsheets.readonly']
SERVICE_ACCOUNT_FILE = 'credenciais-gsheets.json'

def gerar_relatorio_nps(spreadsheet_id, dias=30):
    """Gera relatório NPS consolidado dos últimos N dias"""
    creds = service_account.Credentials.from_service_account_file(
        SERVICE_ACCOUNT_FILE, scopes=SCOPES
    )
    service = build('sheets', 'v4', credentials=creds)

    # Busca respostas do período
    result = service.spreadsheets().values().get(
        spreadsheetId=spreadsheet_id,
        range="'Respostas NPS'!A:F"
    ).execute()

```

```

valores = result.get('values', [])
if len(valores) <= 1:
    return {"erro": "Sem dados suficientes"}

# Filtra por período
data_limite = (datetime.now() - timedelta(days=dias)).isoformat()
respostas = []

for linha in valores[1:]: # Pula header
    if len(linha) >= 4 and linha[0] >= data_limite:
        respostas.append({
            "data": linha[0],
            "telefone": linha[1],
            "nota": int(linha[2]),
            "classificacao": linha[3]
        })

total = len(respostas)
if total == 0:
    return {"erro": "Sem respostas no período"}

promotores = sum(1 for r in respostas if r["classificacao"] == "promotor")
passivos = sum(1 for r in respostas if r["classificacao"] == "passivo")
detratores

```

⑤ Verificação / Gate

```
docker-compose up -d
```

⑥ Feito quando...

- ☐ Você montou o pipeline NPS completo
- ☐ Cada engrenagem está no lugar: disparo automático
- ☐ Mas o que acontece quando a teoria encontra a recepcionista que responde o WhatsApp do paciente pelo celular pessoal?
- ☐ Imagine a cena: é segunda-feira
- ☐ O N8N dispara 45 pesquisas NPS para os pacientes da semana anterior
- ☐ Nos primeiros 10 minutos
- ☐ O Gemini classifica cada uma

⑦ Armadilhas

- (a completar)

Passo 4 — Alertas Executivos

Estágio: Ferramenta · **Origem:** Cap. 4 — Alertas Executivos

① Objetivo do passo

Integrar a planilha finalizada a ferramentas de mensageria (Evolution API) para alertas automáticos no WhatsApp da diretoria.

② Pré-requisito

Passo 3 concluído

③ Entregas

- (a completar)

④ Execução

Instalando a Evolution API com Docker

```
# docker-compose-evolution.yml
# Setup completo da Evolution API para WhatsApp

version: '3.8'

services:
  evolution-api:
    image: atendai/evolution-api:v2.2.3
    container_name: evolution-oficina
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      # Configurações básicas
      - SERVER_URL=http://localhost:8080
      - AUTHENTICATION_API_KEY=sua_api_key_forte_aqui
      - AUTHENTICATION_EXPOSE_IN_FETCH_INSTANCES=true

      # Configurações do Baileys (gratuito)
      - CONFIG_SESSION_PHONE_CLIENT=OficinaImprovisador
      - CONFIG_SESSION_PHONE_NAME=Chrome

      # Webhook para receber mensagens
      - WEBHOOK_GLOBAL_ENABLED=true
      - WEBHOOK_GLOBAL_URL=http://n8n:5678/webhook/whatsapp-recebido
      - WEBHOOK_GLOBAL_WEBHOOK_BY_EVENTS=true

      # Configurações de mensageria
      - CHAT_UPDATE_ENABLED=true
      - CHAT_UPDATE_MINIMAL_CACHE=false

      # Rate limiting (proteção contra bloqueio)
      - DELAY_MESSAGE=3000-7000
      - MAX_MESSAGES_PER_MINUTE=20

      # Timezone
      - TZ=America/Sao_Paulo

volumes:
  - evolution_data:/evolution/instances
```

```

    - evolution_store:/evolution/store

networks:
  - oficina-network

# Redis para cache de sessões (opcional mas recomendado)
redis:
  image: redis:7-alpine
  container_name: evolution-redis
  restart: unless-stopped
  volumes:
    - redis_data:/data
  networks:
    - oficina-network

volumes:
  evolution_data:
  evolution_store:
  redis_data:

networks:
  oficina-network:
    driver: bridge

```

Workflow de alerta de grande venda

```

{
  "name": "Alerta Executivo - Grande Venda",
  "nodes": [
    {
      "parameters": {
        "rule": {
          "interval": [{"field": "cronExpression", "expression": "**/5 * * * *"}]
        }
      },
      "name": "Schedule - A cada 5 minutos",
      "type": "n8n-nodes-base.scheduleTrigger",
      "typeVersion": 1.2,
      "position": [200, 300]
    },
    {
      "parameters": {
        "documentId": {"__rl": true, "value": "SUA_PLANILHA_ID"},
        "sheetName": {"__rl": true, "value": "Vendas Hoje"},
        "filtersUI": {
          "filters": [
            {
              "lookupColumn": "data",
              "lookupValue": "={{ $now.toISOString() }}"
            }
          ]
        }
      },
      "name": "Google Sheets - Vendas do Dia",
      "type": "n8n-nodes-base.googleSheets",
      "typeVersion": 4,
      "position": [430, 300]
    }
  ]
}

```

```

{
  "parameters": {
    "jsCode": "
    // Agrega vendas do dia e identifica grandes vendas
    const vendas = \\
    $input.all();
    const LIMIAR = 5000;
    const grandesVendas = vendas.filter(v =>
    v.json.valor > LIMIAR);
    const totalDia = vendas.reduce((acc, v) => acc +
    (parseFloat(v.json.valor) || 0), 0);
    if (grandesVendas.length === 0) {
      return
      [{ json: { sem_alertas: true, total_dia: totalDia } }];
    }
    // Retorna apenas vendas
    que ultrapassam o limiar
    return grandesVendas.map(v => ({
      json: {
        ...v.json,
        total_dia: totalDia,
        percentual_faturamento: ((v.json.valor / totalDia) *
        100).toFixed(1)
      }
    }));
  },
  "name": "Code - Filtrar Grandes Vendas",
  "type": "n8n-nodes-base.code",

```

Script de teste de envio via Evolution API

```

# teste_evolution_api.py
# Script para testar envio de mensagens via Evolution API

import requests
import json
from datetime import datetime

# Configuração
EVOLUTION_URL = "http://localhost:8080"
API_KEY = "sua_api_key_forte_aqui"
INSTANCE = "oficina-improvisador"

def verificar_conexao():
    """Verifica se a instância WhatsApp está conectada"""
    url = f"{EVOLUTION_URL}/instance/connectionState/{INSTANCE}"
    headers = {"apikey": API_KEY}

    resposta = requests.get(url, headers=headers)

    if resposta.status_code == 200:
        estado = resposta.json()
        print(f"Estado da conexão: {estado.get('state', 'desconhecido')}")
        return estado.get('state') == 'open'
    else:
        print(f"Erro ao verificar conexão: {resposta.status_code}")
        return False

def enviar_mensagem(telefone, mensagem):
    """Envia uma mensagem de texto via WhatsApp"""
    url = f"{EVOLUTION_URL}/message/sendText/{INSTANCE}"
    headers = {
        "Content-Type": "application/json",
        "apikey": API_KEY
    }

    payload = {
        "number": telefone,
        "text": mensagem
    }

    resposta = requests.post(url, headers=headers, json=payload)

```

```
if resposta.status_code == 200 or resposta.status_code == 201:
    resultado = resposta.json()
    print(f"Mensagem enviada com sucesso!")
    print(f" ID: {resultado.get('key', {}).get('id', 'N/A')}")
    print(f" Timestamp: {resultado.get('messageTimestamp', 'N/A')}")
    return True
else:
    print(f"Erro ao enviar: {resposta.status_code}")
    print(f" Resposta: {resposta.text}")
```

Configuração de fallback: Baileys → Cloud API

```
# fallback_envio.py
# Sistema de fallback: tenta Baileys primeiro, cai para Cloud API

import requests
import time
import random

# Configurações
EVOLUTION_URL = "http://localhost:8080"
API_KEY = "sua_api_key_forte_aqui"
INSTANCE_BAILEYS = "oficina-improvisador"
INSTANCE_CLOUD = "oficina-cloud" # Instância Cloud API (pago)

# Cloud API Meta (backup)
CLOUD_API_TOKEN = "seu-token-cloud-api"
CLOUD_PHONE_NUMBER_ID = "seu-phone-number-id"

def enviar_via_baileys(telefone, mensagem):
    """Tenta enviar via Baileys (gratuito)"""
    url = f"{EVOLUTION_URL}/message/sendText/{INSTANCE_BAILEYS}"
    headers = {"Content-Type": "application/json", "apikey": API_KEY}
    payload = {"number": telefone, "text": mensagem}

    try:
        resposta = requests.post(url, headers=headers, json=payload, timeout=10)
        if resposta.status_code in [200, 201]:
            return {"sucesso": True, "canal": "baileys", "custo": 0}
        elif resposta.status_code == 403:
            return {"sucesso": False, "erro": "bloqueado", "canal": "baileys"}
        else:
            return {"sucesso": False, "erro": f"status_{resposta.status_code}", "canal": "baileys"}
    except requests.exceptions.Timeout:
        return {"sucesso": False, "erro": "timeout", "canal": "baileys"}

def enviar_via_cloud_api(telefone, mensagem):
    """Envia via Cloud API Meta (pago, ~\\$0.005/msg)"""
    url = f"https://graph.facebook.com/v18.0/{CLOUD_PHONE_NUMBER_ID}/messages"
    headers = {
        "Authorization": f"Bearer {CLOUD_API_TOKEN}",
        "Content-Type": "application/json"
    }
    payload = {
        "me
```

⑤ Verificação / Gate

```
docker-compose -f docker-compose-evolution.yml up -d
```

⑥ Feito quando...

- ☐ Você montou o sistema de alertas completos: Evolution API configurada
- ☐ A oficina está fechada — cada engrenagem no seu lugar
- ☐ Mas antes de ligar o motor
- ☐ Imagine que você é o diretor financeiro de uma rede de clínicas
- ☐ O sistema de alertas está rodando há duas semanas
- ☐ Na terça-feira às 14h
- ☐ O Gemini formata: “Venda recorde: Implante + Prótese

⑦ Armadilhas

- (a completar)

Checklist Mestre

Passo 1 — O Arsenal Gratuito do Analista

- ☐ Você acabou de montar sua bancada
- ☐ As ferramentas estão afiadas
- ☐ Mas antes de sair ligando tudo
- ☐ Imagine que você é o responsável financeiro de uma rede de 3 clínicas odontológicas
- ☐ O dono pede um relatório consolidado de faturamento do trimestre
- ☐ A armadilha aqui é sutil: a tentação de automatizar tudo de uma vez
- ☐ Você vê o potencial

Passo 2 — O Poder dos Webhooks

- ☐ Você configurou o webhook receiver
- ☐ Agora vamos ver o que acontece quando a teoria encontra a prática —
- ☐ Imagine que você é o analista de uma clínica odontológica que fatura R\$ 180 mil por mês
- ☐ O dono quer ser alertado no WhatsApp sempre que uma venda de R\$ 5 mil ou mais acontecer
- ☐ Você monta o workflow: planilha alimenta webhook
- ☐ O primeiro dia em produção revela a verdade
- ☐ A recepcionista cadastra 3 vendas simultâneas

Passo 3 — Automação Pós-Venda na Prática

- ☐ Você montou o pipeline NPS completo
- ☐ Cada engrenagem está no lugar: disparo automático
- ☐ Mas o que acontece quando a teoria encontra a recepcionista que responde o WhatsApp do paciente pelo celular pessoal?
- ☐ Imagine a cena: é segunda-feira
- ☐ O N8N dispara 45 pesquisas NPS para os pacientes da semana anterior
- ☐ Nos primeiros 10 minutos
- ☐ O Gemini classifica cada uma

Passo 4 — Alertas Executivos

- ☐ Você montou o sistema de alertas completos: Evolution API configurada
- ☐ A oficina está fechada — cada engrenagem no seu lugar
- ☐ Mas antes de ligar o motor
- ☐ Imagine que você é o diretor financeiro de uma rede de clínicas
- ☐ O sistema de alertas está rodando há duas semanas
- ☐ Na terça-feira às 14h
- ☐ O Gemini formata: "Venda recorde: Implante + Prótese